



INSTITUTO SUPERIOR TÉCNICO

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

ORGANIZAÇÃO DE COMPUTADORES

LEIC

First Lab Assignment: System Modeling and Profiling

Version 1.2

2025/2026

1 Introduction

The goal of this assignment is twofold: (i) to determine the characteristics of a computer's caches, and (ii) to leverage this knowledge to optimize the performance of a given program. For this task, students will use a performance analysis tool that provides direct access to hardware performance counters available on most modern microprocessors. The tool employed is the standard Application Programming Interface (API), PAPI [1].

In the rest of this section, we briefly introduce PAPI and describe the targeted computer platform and development environment. In Section 3, we present the procedure for modeling the L1 and L2 caches of the target platform (Subsection 3.1), and provide a guide for analyzing the performance of a matrix-multiply code segment and optimizing it based on the characteristics of the L2 cache of the target architecture (Subsection 3.2).

1.1 Targeted Platform and Development Environment

IMPORTANT: This assignment **must** be performed on the computers of your lab classroom. **Consult your lab instructor if you have any doubts.** These computers share similar hardware characteristics, and any of them may be used as the target platform. Since this work is hardware-dependent, using a computer with different hardware could lead to unexpected results and thus invalidate your work. Therefore, always use the same lab environment.

To set up the development environment, you will need the PAPI library and a set of auxiliary program files. This material is provided in the package **lab1_kit.zip**, available on the course website. After downloading and extracting the package on one of the lab computers, you must build PAPI. To do this, navigate to the source code directory **papi-X.X.X/src** and compile by running:

```
./configure
```

```
Make
```

This process generates a set of helper tools in the **src/utils/** directory and creates the PAPI library **papilib.a**. Among the tools, `papi_availis` is especially useful for checking which PAPI events are supported on the target platform. The library will later be linked to the auxiliary programs described in the following sections.

2 First steps

To help determine the characteristics of the labs computer's caches, the following exercises will help you estimate cache parameters from small C applications.

The first step to get acquainted with the procedure is to determine only the size of the cache using a small C application on a (known) machine, such as the code you have analyzed on lab exercise VI.3. This C code is a simplified version of the following programs in this assignment. Basically, it iterates over an array to determine the cache size.

To guarantee that you measure the time accurately, please use the source code available in the lab kit (file `spark.c`).

In order to perform the evaluation you should go to your lab to access the cache size by running the application there. You may want to repeat the evaluation of the elapsed time a few times to achieve statistical significance. You should table the relevant results for different cache sizes on the response sheet and make a conclusion regarding the cache size. You can calculate more measures before the output, examine the final part of the source code file.

1. What is the cache capacity of the computer you evaluated? Please justify.

To discover the other cache parameters, you're going to modify the C application, so that it generates different data access patterns. Please spend a few minutes analyzing the modifications to the source code.

```
for(size_t cache_size = CACHE_MIN; cache_size < CACHE_MAX; cache_size = 2*cache_size)
{ for(size_t stride = 1; stride <= cache_size/2; stride = 2*stride){ limit =
  cache_size - stride + 1;
  for(ssize_t repeat = 0; repeat < N_REPETITIONS * stride; repeat++)
  { for(index = 0; index < limit; index += stride) { array[index] =
    array[index] + 1;
  }
}
}
```

The meaning of each variable is the following:

array[] an arbitrary large array that will be repeatedly accessed to measure the cache miss pattern;

cache_size value of the cache size under test; all cache sizes given by integer powers of 2, between `CACHE_MIN = 8kB` and `CACHE_MAX = 64kB` should be considered;

stride states how many entries are being skipped at each access; for example, if the stride is 4, entries 0, 4, 8, 12, ... in the array are being accessed, while entries 1, 2, 3, 5, 6, 7, 9, 10, 11, ... are skipped;

limit the largest address that will be accessed for the cache size and access pattern under test;

repeat denotes the number of times that each access pattern will be repeated in an array.

This code segment was executed on an arbitrary machine. By varying the stride parameter and testing across different array sizes (ranging from `ARRAY_MIN = 4kB` to `ARRAY_MAX = 4MB`), the results from Figure 1 were obtained. Using solely the **information from the graph** answer the following questions:

2. What is the cache capacity of the computer?
3. What is the size of each cache block?
4. What is the L1 cache miss penalty time?

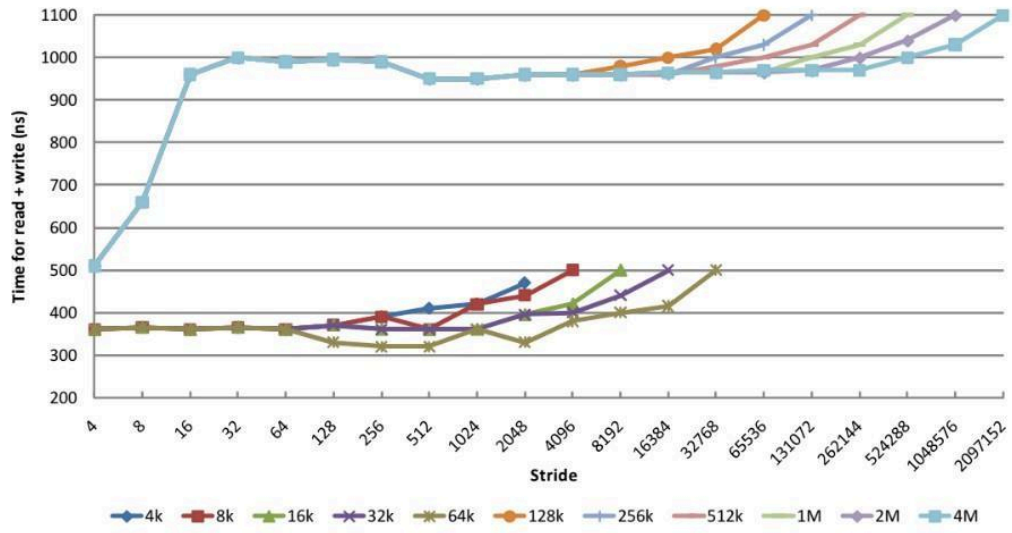


Figure 1: Variation of the cache access time with the adopted *stride* value for different array sizes.

3 Assignment

3.1 Modeling Computer Caches

In the first part of this assignment, the goal is to model the characteristics of the L1 data cache and L2 cache of the targeted computer platform. Next, we provide instructions for performing this analysis. Use the forms at the end to answer the questions below.

3.1.1 Modeling the L1 Data Cache

The methodology to experimentally model the L1 data cache consists in considering the total amount of data cache misses during the execution of the following code sequence of program `cm1.c`, similar to the program in Section 2. This program can be found in the package `lab1_kit.zip`.

```
for(size_t cache_size = CACHE_MIN; cache_size < CACHE_MAX; cache_size = 2*cache_size)
{ for(size_t stride = 1; stride <= cache_size/2; stride = 2*stride){ limit =
  cache_size - stride + 1;
  for(ssize_t repeat = 0; repeat < N_REPETITIONS * stride; repeat++)
    { for(index = 0; index < limit; index += stride) { array[index] =
      array[index] + 1; }
    }
  }
}
```

- a) Change to directory `cm1/`, in the package `lab1_kit.zip`, and analyze the code of the program `cm1.c`. Identify its source code with the program described above.

What are the processor events that will be analyzed during its execution? Explain their meaning.

- b) Compile the program `cm1.c` using the provided `Makefile` and execute `cm1`. **Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).**

NOTE: A fast sketch of these plots can be drawn in your computer by running the following commands:

```
./cm1 > cm1.out
./cm1_proc.sh
```

NOTE 2: You can draw these tables and plots on your computer, print, and attach to the report. You do not have to fill them by hand on the printed report.

NOTE 3: You may need to mark the script as executable before being able to run it.

- c) By analyzing the obtained results:

- **Determine the size of the L1 data cache. Justify your answer.**
- **Determine the block size adopted in this cache. Justify your answer.**
- **Characterize the associativity set size adopted in this cache. Justify your answer.**

3.1.2 Modeling the L2 Cache

In this part of the assignment, the goal is to experimentally model the characteristics of the L2 cache of the targeted computer platform. To analyze the computer's L2 cache, we will use the same methodology that was introduced in the previous section to model the L1 data cache.

- a) Modify the program `cm1.c` in order to analyze the characteristics of the L2 cache. (Hint: use the event `PAPI_L2_DCM`.) **Describe and justify the changes introduced in this program.**
- b) Compile the program `cm1.c`, execute `cm1`, and **plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.**

By analyzing the obtained results:

- **Determine the size of the L2 cache. Justify your answer.**

- Determine the block size adopted in this cache. Justify your answer.
- Characterize the associativity set size adopted in this cache. Justify your answer.

3.2 Profiling and Optimizing Data Cache Accesses

Often, programmers wishing to improve their programs' performance focus their attention on how the programs affect the computer's caches. In the following, it will be analyzed how simple code changes can help to improve that performance for a matrix multiplication application.

Consider a simple matrix multiplication application, operating on two square matrices of $N \times N$

16-bit integer elements, with $N = 1024$. From a mathematical point of view, given two matrices **A** and **B**, with elements a_{ij} and b_{ij} such that $0 \leq i, j < N$, the product matrix **C** is defined as:

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{i(N-1)}b_{(N-1)j}$$

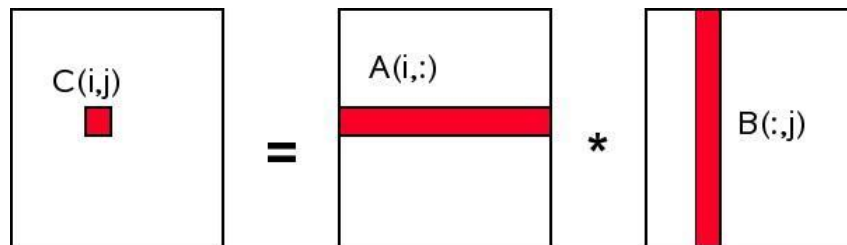


Figure 2: Straightforward matrix multiplication.

3.2.1 Straightforward implementation

A straight-forward C implementation of Eq. 1 can look like this:

```
for (i = 0; i < N; ++i) { for (j = 0; j < N;
++j){ for (k = 0; k < N; ++k){ res[i][j] +=
mul1[i][k] * mul2[k][j]; }
}
}
```

The two input matrices are `mul1` and `mul2`. The result matrix `res` is assumed to be initialized to all zeroes.

The provided program `mm1.c` includes this code sequence and all the necessary initialization steps, as well as the set of statements that are required in order to profile its execution using the PAPI toolbox.

- Change to directory `mm1/` and analyze the code of the program `mm1.c`. Identify its source code with the program described above.
What is the total amount of memory that is required to accommodate each of these matrices?
- Compile the source file `mm1.c` using the provided `Makefile` and execute it. **Fill the table with the obtained data.**
- Evaluate the resulting L1 data cache *Hit-Rate*.**

3.2.2 First Optimization: Matrix transpose before multiplication [2]

By analyzing the obtained results, it can be observed that such a straightforward implementation suffers from a severe penalty in what concerns the amount of L2 cache misses resulting from its access pattern. In fact, while the `mul1` matrix is accessed sequentially, the inner loop advances the row number of `mul2` (see Fig. 2), meaning successive accesses to far away memory positions.

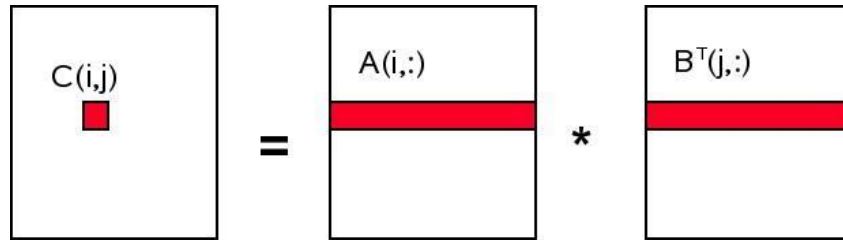


Figure 3: Transposed matrix multiplication.

One possible remedy to attenuate such a problem is based on matrix transposition. In fact, since each matrix element is accessed multiple times, it might be worthwhile to rearrange (“transpose,” in mathematical terms) the second matrix `mul2` before using it (see Fig. 3):

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{jk}^T = a_{i1} b_{j1}^T + a_{i2} b_{j2}^T + \dots + a_{i(N-1)} b_{j(N-1)}^T$$

After the preliminary transposition step, both matrices may be iterated sequentially. As far as the C code is concerned, it now looks like this:

```
int16_t tmp[N][N];

// transposition for (i = 0; i
< N; ++i) { for (j = 0; j < N;
++j){ tmp[i][j] = mul2[j][i];
}
}

// multiplication for (i = 0; i <
N; ++i) { for (j = 0; j < N; ++j){
for (k = 0; k < N; ++k) {
    res[i][j] += mul1[i][k] * tmp[j][k];
}
}
}
```

Variable `tmp` is a temporary array to store the transposed matrix. One direct consequence of this optimization is that it now requires additional access to the data memory. Hopefully, this extra cost can be easily recovered, since the 1024 non-sequential accesses per column are usually much more expensive.

- a) Change to directory `mm2/` and analyze the code of the program `mm2.c`. Identify its source code with the program described above. Compile this program using the provided `Makefile` and execute it. **Fill the table with the obtained data.**
- b) **Evaluate the resulting L1 data cache *Hit-Rate*.**
- c) Change the code in the program `mm2.c` in order to include the matrix transposition in the execution time. Compile this program using the provided `Makefile` and execute it.
Fill the table with the obtained data. Comment on the obtained results when including the matrix transposition in the execution time.
- d) **Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedups.**

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

Despite the good results that may be obtained with the matrix transposition method, in many applications this approach can not be applied, either because the matrix is too large or the available memory is too small. Hence, other alternatives, which do not require the extra copy procedure, should be studied.

The search for an alternative processing scheme should start with a close examination of the involved math and the operations performed by the original implementation. Trivial math knowledge shows that the order of the several additions to obtain each element of the result matrix is irrelevant, as long as each addend appears exactly once. This understanding will lead to solutions which reorder the additions performed in the inner loop of the original code.

According to the original algorithm, the adopted order to access the elements of matrix `mul2` is: (0,0), (1,0), ... , (N-1,0), (0,1), (1,1), ... Although the elements (0,0) and (0,1) are in the same cache line, by the time the inner loop completes one round, this cache line has long been evicted. For this example, each round of the inner loop requires, for each of the three matrices, 1024 cache lines, which is much more than what is available in most processors' caches.

One possible solution is to simultaneously handle more than one iteration of the middle loop, while executing the inner loop. In this case, several values which are guaranteed to be in cache will be used, thus contributing to a reduction of the L2 cache miss-rate. Hence, to maximize the speedup provided by this technique, it is necessary to adapt the dimension of the sub-matrix under processing to the cache block size, by taking into account the size of each matrix element. As a hypothetical example, considering that a `short` operand occupies 2-Bytes, this means that a 64-Byte cache block will accommodate 32 matrix elements, thus defining the optimal size for the sub-matrix line to be 32 (see Fig. 4).

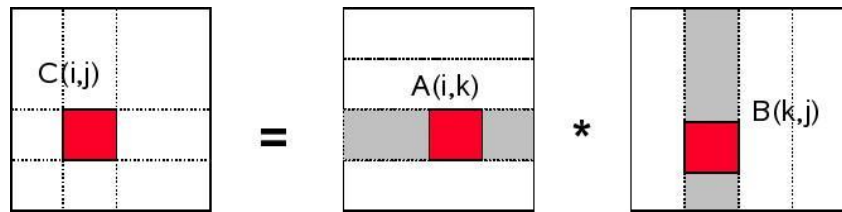


Figure 4: Blocked matrix multiplication.

As far as the C code is concerned, it now looks like this:

```
#define SUB_MATRIX_SIZE (CACHE_LINE_SIZE / sizeof (short))
for (size_t i = 0; i < N; i += SUB_MATRIX_SIZE) {
    for (size_t j = 0; j < N; j += SUB_MATRIX_SIZE) {
        for (size_t k = 0; k < N; k += SUB_MATRIX_SIZE) {
            for (size_t inner_i = 0; inner_i < SUB_MATRIX_SIZE; ++inner_i) {
                for (size_t inner_k = 0; inner_k < SUB_MATRIX_SIZE; ++inner_k) {
                    for (size_t inner_j = 0; inner_j < SUB_MATRIX_SIZE; ++inner_j) {
                        res[i + inner_i][j + inner_j] +=
                            factor1[i + inner_i][k + inner_k] *
                                factor2[k + inner_k][j + inner_j];
                    }
                }
            }
        }
    }
}
```

The most visible change is that the code has six nested loops now. The outer loops iterate with intervals of `SUB_MATRIX_SIZE` (the cache line size `CACHE_LINE_SIZE` divided by `sizeof(short)`). This divides the matrix multiplication into several smaller problems which can be handled with more cache locality. The inner loops iterate over the missing indexes of the outer loops. There are, once again, three loops. The `k2` and `j2` loops are in a different order. This is done because, in the actual computation, only one expression depends on `k2` but two depend on `j2`.

- a) Change to directory `mm3/` and analyze the code of the program `mm3.c`. Identify its source code with the program described above.

Change the program source code in order to comply with the algorithm parameterization (sub-matrix line size) with the block size (CLS) that was determined in Section 3.1.

How many matrix elements can be accommodated in each cache line?

- b) Compile this program using the provided `Makefile` and execute it. Fill the table with the obtained data. Evaluate the resulting L1 data cache *Hit-Rate*.
- c) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.
- d) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.

If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement?

(Hint: study the hit-rates of the L2 cache for both implementations; You may use the following PAPI events PAPI_L2_DCH (or PAPI_L2_DCM) and PAPI_L2_DCA. Run `papi_avail` to check for available events and understand their meaning.)

References

- [1] Performance Application Programming Interface (PAPI). Webpage. "<http://icl.cs.utk.edu/papi>", December 2008.
- [2] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., November 2007.
- [3] *PAPI User's Guide*.
- [4] *PAPI Programmer's Reference*.

Second Lab Assignment: System Modeling and Profiling

STUDENTS IDENTIFICATION:

Number:	Name:
1110179	Renato Filipe Antunes Neto
1109605	Sebastião Gomes Sequeira Lemos de Mendonça
1109881	Tomás Martins Ferreira

2 First steps

Please justify all your answers with values from the experiments.

1. What is the cache capacity of the computer you used (please write the workstation name)?

Array Size	8 KiB	16 KiB	32 KiB	64 KiB	128 KiB	256 KiB
t2-t1	0.001987	0.004211	0.008654	0.049981	0.102317	0.210659
# accesses a[i]	1638400	3276800	6553600	13107200	26214400	52428800
# mean access time	1.212681	1.285158	1.320541	3.813221	3.903082	4.018010

Até ao array size de 32 KiB o tempo médio de acesso foi praticamente igual. No entanto, a partir do array de 64 KiB observamos uma subida do tempo médio de 1.3 para 3.8. Isto indica que o array de 64 KiB já não cabe neste nível de cache (L1) e teve de trocar para o nível seguinte, provocando um aumento do tempo médio de acesso. Sendo assim podemos concluir que o tamanho da cache é 32 KiB. (foi usado o PC i5-033)

Consider the data presented in Figure 1. Answer the following questions (2, 3, 4) about the machine used to generate that data.

2. What is the cache capacity?

No gráfico da figura 1 observamos que o tempo de read+write mantém-se praticamente igual até ao tamanho de array de 64 KiB. Já para arrays de tamanho superior a 64 KiB é possível observar uma subida abrupta do tempo de read+write. Isto indica que a cache já não tem espaço para esses tamanhos de array e portanto a capacidade da cache é de 64 KiB.

3. What is the size of each cache block?

Para analisar o tamanho de cada bloco da cache, temos de analisar a curva de um array que não cabe na cache para verificar com que stride todos os acessos resultam em miss. (Num array de tamanho menor do que a capacidade da cache vamos sempre ter hits independentemente do stride). Sendo assim podemos verificar que o tempo de acesso à medida que aumentamos o stride também aumenta até ficar constante (alcança 100% misses). Este valor constante é observado a partir de um stride de 16 B. Isso indica que, quando o stride é 16 B saltamos de bloco em bloco e por isso todos os acessos dão miss. Se o stride for menor que 16 B vamos ter acesso dentro dos blocos que dão hit. Concluimos assim que o tamanho de cada bloco da cache é 16 B.

4. What is the L1 cache miss penalty time?

Para calcular o miss penalty time vamos utilizar o miss time (tempo de acesso em que o nosso programa irá dar somente misses) e subtrair o hit time (tempo em que o hit rate é máximo). No gráfico verificamos que o todos os acessos dão miss para arrays maiores do que o tamanho da cache e strides maiores do que o tamanho do bloco, ou seja, o miss time é aproximadamente 1000 ns. Para tamanhos de array menores que a capacidade da cache, praticamente todos os acessos dão hit, logo hit time é aproximadamente 370 ns.

Desta maneira o miss penalty é $1000 - 370 = 630$ ns.

3 Assignment

3.1.1 Modeling the L1 Data Cache

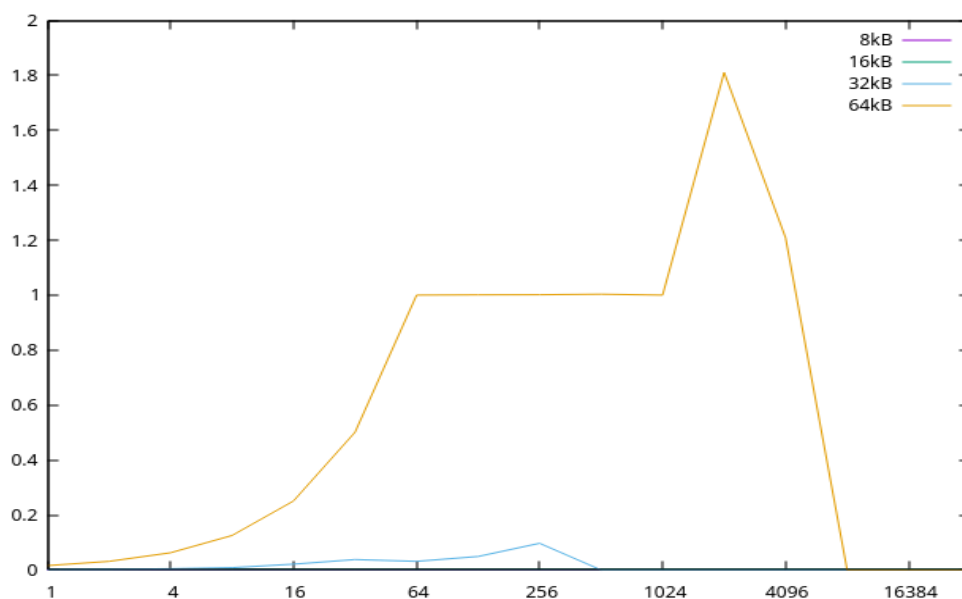
- a) What are the processor events that will be analyzed during its execution? Explain their meaning.

Durante esta execução vamos analisar o número de misses da cache L1. Este evento de que se fala é PAPI_L1_DCM em que DCM representa Data Cache Misses. A leitura dos eventos é inicializada antes do Loop com o PAPI_start(EventSet), a contagem de ciclos com PAPI_get_real_cyc() e PAPI_get_real_usec() para o tempo de execução. No fim da execução imprimimos para o stdout o número médio de misses por iteração, o tempo médio de acesso à memória e o número médio de ciclos (respetivamente $(\text{float})(\text{values}[0]) / \text{n_iterations}$, $(\text{float})(\text{end_usec} - \text{start_usec}) / \text{n_iterations}$ e por fim $(\text{float})(\text{end_cycles} - \text{start_cycles}) / \text{n_iterations}$).

- b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).
Note that, you may fill these tables and graphics (as well as the following ones in this report) on your computer and submit the printed version.

Array Size	Stride	Avg Misses	Avg Cycl Time
8kBytes	1	0.000329	0.003898
	2	0.000106	0.003863
	4	0.000194	0.003946
	8	0.000039	0.004054
	16	0.000032	0.003690
	32	0.000095	0.003571
	64	0.000029	0.003644
	128	0.000020	0.003035
	256	0.000013	0.002676
	512	0.000010	0.002498
	1024	0.000010	0.002372
	2048	0.000008	0.002364
	4096	0.000007	0.002372
16kBytes	1	0.000121	0.002473
	2	0.000073	0.002479
	4	0.000098	0.002511
	8	0.000073	0.002563
	16	0.000074	0.002540
	32	0.000086	0.002599
	64	0.000085	0.002733
	128	0.000041	0.002521
	256	0.000022	0.002594
	512	0.000010	0.002728
	1024	0.000005	0.002282
	2048	0.000002	0.002328
	4096	0.000002	0.002451
	8192	0.000002	0.002360

Array Size	Stride	Avg Misses	Avg Cycl Time
32kBytes	1	0.001431	0.002473
	2	0.002426	0.002479
	4	0.004487	0.002494
	8	0.008461	0.002610
	16	0.021173	0.002570
	32	0.037644	0.002590
	64	0.031459	0.002740
	128	0.049006	0.002492
	256	0.096933	0.002543
	512	0.000050	0.002591
	1024	0.000026	0.002728
	2048	0.000008	0.002390
	4096	0.000003	0.002571
	8192	0.000002	0.002449
64kBytes	16384	0.000002	0.002361
	1	0.015656	0.002468
	2	0.031314	0.002479
	4	0.062686	0.002475
	8	0.125369	0.002494
	16	0.250734	0.002511
	32	0.501328	0.002553
	64	1.000408	0.002895
	128	1.000955	0.002846
	256	1.001848	0.002767
	512	1.003393	0.002783
	1024	1.000003	0.002941
	2048	1.810900	0.005315
	4096	1.209148	0.008259
	8192	0.000005	0.002571
	16384	0.000002	0.002548
	32768	0.000002	0.002364



c) By analyzing the obtained results:

- Determine the **size** of the L1 data cache. Justify your answer.

Ao observar o gráfico obtido, determinamos que o tamanho da cache de dados L1 é 32kB. Para arrays de 8 kB, 16kB e 32kB observamos que o número de misses é igual, no entanto quando testamos um array de 64kB observamos um aumento substancial do número de misses, ou seja, esse array ultrapassou o tamanho da cache.

- Determine the **block size** adopted in this cache. Justify your answer.

Para determinar o tamanho do bloco, analisamos a curva do array de 64 kB, e observamos que os misses passam a ficar constantes para um stride de 64B. Ou seja, a partir desse stride os nossos acessos vão ser sempre misses uma vez que saltamos de bloco em bloco.

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

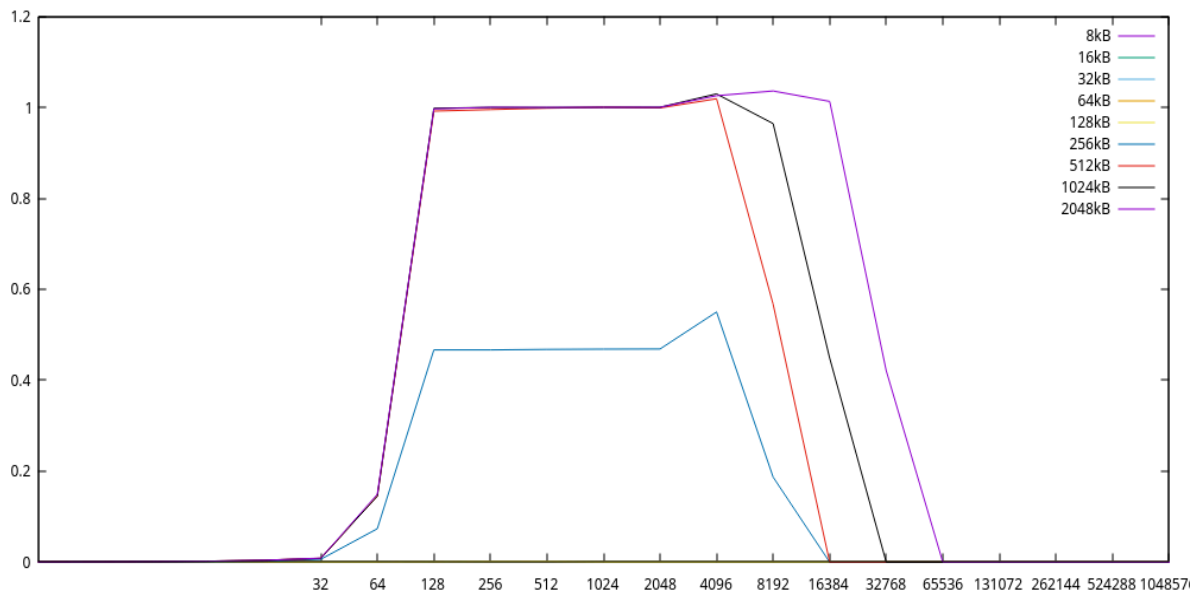
Para descobrir o número de vias de associatividade temos de analisar o comportamento do gráfico entre o stride 4096 B e 8192 B para um array de tamanho maior que a capacidade da cache. Neste intervalo observamos que o número de misses diminui drasticamente, passando a ser semelhante ao número de misses para arrays menores que a capacidade da cache. Isto indica que para um stride de 4096 B ainda existem conflitos entre blocos para endereços referentes ao mesmo índice. No caso do array de 64 kB, existem $64 \text{ kB} / 4 \text{ kB} = 16$ endereços que vão para o mesmo índice da cache e que causam conflitos. Já no caso do stride de 8192 B, observamos que existem $64 \text{ kB} / 8 \text{ kB} = 8$ endereços referentes ao mesmo índice de cache, no entanto, estes não geram conflitos e por isso o número de misses é menor. Isto indica que cada índice da cache tem espaço para 8 blocos em simultâneo (8-way set associative). Sendo assim temos $32 \text{ kB} / (64 \text{ B} \times 8) = 64$ sets e cada set tem $64 \text{ B} \times 8 = 512 \text{ Bytes}$.

3.1.2 Modeling the L2 Cache

- a) Describe and justify the changes introduced in this program.

Para analisarmos agora os misses da cache L2 temos de alterar o evento de `PAPI_add_event(EventSet, PAPI_L1_DCM)` para `PAPI_add_event(EventSet, PAPI_L2_DCM)`. Além disso temos de aumentar a range de tamanho dos arrays que vamos testar alterando o `CACHE_MAX` para $(2048 * 1024)$ uma vez que a cache L2 normalmente é maior do que a L1.

- b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.



- c) By analyzing the obtained results:

- Determine the **size** of the L2 cache. Justify your answer.

O tamanho da cache L2 é determinada observando para que tamanho de arrays a diferença de misses é maior. Neste caso observamos um salto do array de 128 kB para o array de 256kB no entanto este não é o maior salto e possivelmente deve-se a outras razões como por exemplo a não separação da cache de instruções e de dados em L2. Sendo assim é a partir do array de 512kB inclusive que a capacidade da cache é ultrapassada dando origem a um aumento enorme do número de misses. Assim verificamos que o tamanho de L2 é de 256 KB

- Determine the **block size** adopted in this cache. Justify your answer.

O tamanho dos blocos de L2 é determinado vendo o stride a partir do qual o número de misses se mantém constante para um array size maior do que a capacidade da cache, tal como foi determinado para a cache L1. Desta forma o tamanho de cada bloco é 128B.

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

Usando a mesma lógica da cache L1, observamos que para um array maior que a capacidade da cache (por exemplo 512 kB), observamos que há uma diminuição do número de misses entre os strides 8192 B e 16384 B. Ou seja, quando o stride é 16384 B, temos $512 \text{ kB} / 16 \text{ kB} = 32$ endereços para o mesmo índice de cache sem causar conflitos (avg misses baixo). Isto indica que cada índice da cache tem espaço para 32 blocos em simultâneo (32-way set associative). Sendo assim temos $256 \text{ kB} / (128 \text{ B} \times 32) = 64$ sets e cada set tem $128 \text{ B} \times 32 = 4096$ Bytes.

3.2 Profiling and Optimizing Data Cache Accesses

3.2.1 Straightforward implementation

- a) What is the total amount of memory that is required to accommodate each of these matrices?

Observando o código de mm1.c, verificamos que as matrizes são de $N \times N$, com $N=512$ e que cada elemento da matriz é um inteiro de 16 bits, ou seja 2 bytes. Sendo assim, a quantidade de memória total necessária para acomodar cada matriz é $512 \times 512 \times 2 = 524288$ KiB.

- b) Fill the following table with the obtained data.

Total number of L1 data cache misses	135.077500×10^6
Total number of load / store instructions completed	536.872758×10^6
Total number of clock cycles	816.707168×10^6
Elapsed time	0.248056 seconds

- c) Evaluate the resulting L1 data cache *Hit-Rate*:

$\text{Hit-Rate} = (\text{n}^\circ \text{ hits} / \text{n}^\circ \text{ of accesses}) \times 100 = ((\text{n}^\circ \text{ of accesses} - \text{n}^\circ \text{ misses}) / \text{n}^\circ \text{ of accesses}) \times 100$
 $= ((536.872758 \times 10^6 - 135.077500 \times 10^6) / 536.872758 \times 10^6) \times 100 =$
 $= 0.7484 \times 100 = \mathbf{74.84\%}$

3.2.2 First Optimization: Matrix transpose before multiplication [2]

- a) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.505649×10^6
Total number of load / store instructions completed	537.938482×10^6
Total number of clock cycles	767.694335×10^6
Elapsed time	0.233170 seconds

- b) Evaluate the resulting L1 data cache *Hit-Rate*:

$\text{Hit-Rate} = (\text{n}^\circ \text{ hits} / \text{n}^\circ \text{ of accesses}) \times 100 = ((\text{n}^\circ \text{ of accesses} - \text{n}^\circ \text{ misses}) / \text{n}^\circ \text{ of accesses}) \times 100$
 $= ((537.938482 \times 10^6 - 4.505649 \times 10^6) / 537.938482 \times 10^6) \times 100 =$
 $= 0.9916 \times 100 = \mathbf{99.16\%}$

- c) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.476898×10^6
Total number of load / store instructions completed	537.94028×10^6
Total number of clock cycles	769.823792×10^6
Elapsed time	0.233817 seconds

Comment on the obtained results when including the matrix transposition in the execution time:

Ao incluir a transposição de matrizes no tempo de execução notamos um ligeiro aumento do tempo, do número de instruções load/store e ciclos de relógio e uma ligeira diminuição do número de misses na cache L1. No entanto, estas alterações não alteram o desempenho de forma relevante.

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedups.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm2}} - \text{HitRate}_{\text{mm1}}: 0.9916 - 0.7484 = 0.2432$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm1}} / \#\text{Clocks}_{\text{mm2}}: 816.707168 / 767.694335 = 1.06$
$\text{Speedup}(\text{Time}) = \text{Time}_{\text{mm1}} / \text{Time}_{\text{mm2}}: 0.248056 / 0.233170 = 1.06$
Comment: Todos os resultados obtidos indicam que o método usado em mm2 é mais eficiente do que o usado em mm1. Ou seja, transpor a matriz antes da multiplicação é melhor do que a abordagem simples de multiplicação. O motivo para o método mm2 obter melhores resultados deve-se ao facto de que multiplicamos linhas de uma matriz por linhas de outra em vez de colunas, ou seja, a cache carrega menos blocos uma vez que os elementos das linhas da matriz estão alinhados na cache. Isto faz com que o número de misses seja menor e por sua vez tempo de execução.

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

- a) How many matrix elements can be accommodated in each cache line?

Sendo que cada bloco da cache ocupa 64B e os elementos da Matriz são inteiros de 2 bytes(16 bits), cada linha pode acomodar $64/2$ inteiros, ou seja, 32 inteiros

- b) Fill the following table with the obtained data.

Total number of L1 data cache misses	3.653647×10^6
Total number of load / store instructions completed	537.802878×10^6
Total number of clock cycles	337.636997×10^6
Elapsed time	0.102550 seconds

- c) Evaluate the resulting L1 data cache *Hit-Rate*:

$\text{Hit-Rate} = (\text{n}^\circ \text{ hits} / \text{n}^\circ \text{ of accesses}) \times 100 = ((\text{n}^\circ \text{ of accesses} - \text{n}^\circ \text{ misses}) / \text{n}^\circ \text{ of accesses}) \times 100$
 $= ((537.802878 \times 10^6 - 3.653647 \times 10^6) / 537.802878 \times 10^6) \times 100 =$
 $= 0.9932 \times 100 = \mathbf{99.32\%}$

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm1}}: 0.9932 - 0.7484 = 0.2448$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm1}} / \#\text{Clocks}_{\text{mm3}}: 816.707168 / 337.636997 = 2.42$
Comment: Esta segunda otimização (mm3) obteve um melhor desempenho relativamente a mm1 tanto no hit rate como nos clock cycles, o que nos deu um speedup de 2.42. Isto indica que a abordagem de dividir a multiplicação de matrizes em problemas mais pequenos melhora o tempo de execução uma vez que beneficia a localidade da cache, ao contrário da abordagem mm1.

- e) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup. If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement? (Hint: study the hit-rates of the L2 cache for both implementations;)

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm2}}: 0.9932 - 0.9916 = 0.0016$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm2}} / \#\text{Clocks}_{\text{mm3}}: 767.694335 / 337.636997 = 2.27$
Comment: A diferença de hit rate entre a abordagem mm3 e mm2 não é significativa (0.0016), no entanto obtivemos um speedup de 2.27, ou seja, o número de clock cycles para mm3 diminuiu. Ao dividir a matriz original em blocos menores, cada bloco cabe melhor nos níveis de cache (especialmente em L2). Isso permite reutilizar os dados da cache de maneira mais eficiente, reduzindo a necessidade de carregar blocos, diminuindo assim o tempo de execução.

3.2.3 Comparing results against the CPU specifications

Now that you have characterized the cache on your lab computer, you are going to compare it against the manufacturer's specification. For this you can check the device's datasheet, or make use of the command `lscpu`. Comment on the results.

Trabalhamos no PC i5-033. O que nos interessa do output do comando é:
 L1d e L1i: 128 KiB (4 instances) -> 128 KiB / 4 = 32 kiB por instância
 L2: 1 MiB (4 instances) -> 1MiB / 4 = 256 kiB por instância
 L3: 6 MiB (1 instance)

O resultado que obtivemos foi de 32 KiB para a cache L1 e 256 KiB para a cache L2, ou seja, podemos averiguar que os nossos resultados estão corretos. Concluimos assim, que é possível identificar as características de um processador analisando seu desempenho dado um programa específico, através de métricas como hit rate, miss rate, ciclos de clock, tempo de execução e número de acessos à memória. A partir desses dados, é possível inferir detalhes sobre a arquitetura de um processador.

A PAPI - Performance Application Programming Interface

The PAPI project [1] specifies a standard Application Programming Interface (API) for accessing hardware performance counters available in most modern microprocessors. These counters exist as a small set of registers that count *Events*, defined as occurrences of specific signals related to the processor's function (such as cache misses and floating point operations), while the program executes on the processor. Monitoring these events may have a variety of uses in the performance analysis and tuning of an application, since it facilitates the correlation between the source/object code structure and the efficiency of the actual mapping of such code to the underlying architecture. Besides performance analysis, and hand tuning, this information may also be used in compiler optimization, debugging, benchmarking, monitoring and performance modeling.

PAPI has been implemented on a number of different platforms, including: Alpha; MIPS R10K and R12K; AMD Athlon and Opteron; Intel Pentium II, Pentium III, Pentium M, Pentium IV, Itanium 1 and Itanium 2; IBM Power 3, 4 and 5; Cell; Sun UltraSparc I, II and II, etc.

Although each processor has a number of events that are native to that specific architecture, PAPI provides a software abstraction of these architecture-dependent *Native Events* into a collection of *Preset Events*, also known as *predefined events*, that define a common set of events deemed relevant and useful for application performance tuning. These events are typically found in many CPUs that provide performance counters. They give access to the memory hierarchy, cache coherence protocol events, cycle and instruction counts, functional unit, and pipeline status. Hence, preset events may be regarded as mappings from symbolic names (PAPI preset name) to machine specific definitions (native countable events) for a particular hardware resource. For example, Total Cycles (in user mode) is mapped into PAPI_TOT_CYC. Some presets are derived from the underlying hardware metrics. For example, Total L1 Cache Misses (PAPI_L1_TCM) is the sum of L1 Data Misses and L1 Instruction Misses on a given platform. The list of preset and native events that are available on a specific platform can be obtained by running the commands `papi_avail` and `papi_native_avail`, both provided by the papi source distribution.

Besides the standard set of events for application performance tuning, the PAPI specification also includes both a high-level and a low-level sets of routines for accessing the counters. The high level interface consists of eight functions that make it easy to get started with PAPI, by simply providing the ability to start, stop, and read sets of events. This interface is intended for the acquisition of simple but accurate measurement by application engineers [3, 4]:

- `PAPI_num_counters` – get the number of hardware counters available on the system;
- `PAPI_flops` – simplified call to get Mflops/s (floating point operation rate), real and processor time;
- `PAPI_ipc` – gets instructions per cycle, real and processor time;
- `PAPI_accum_counters` – add current counts to array and reset counters;
- `PAPI_read_counters` – copy current counts to array and reset counters;
- `PAPI_start_counters` – start counting hardware events;
- `PAPI_stop_counters` – stop counters and return current counts.

The following is a simple code example of using the high-level API [3, 4]:

```
#include <papi.h>

#define NUM_FLOPS 10000
#define NUM_EVENTS 1

int main(){
    int Events[NUM_EVENTS] = {PAPI_TOT_INS};
    long_long values[NUM_EVENTS];

    /* Start counting events */
    if (PAPI_start_counters(Events, NUM_EVENTS) !=
        PAPI_OK)
```

```

        handle_error(1);

do_some_work();

/* Read the counters */
if (PAPI_read_counters(values, NUM_EVENTS) != PAPI_OK)
    handle_error(1);
printf("After reading the counters: %lld\n", values[0]);

do_some_work();

/* Add the counters */
if (PAPI_accum_counters(values, NUM_EVENTS) !=
PAPI_OK)
    handle_error(1);
printf("After adding the counters: %lld\n", values[0]);

do_some_work();

/* Stop counting events */
if (PAPI_stop_counters(values, NUM_EVENTS) != PAPI_OK)
    handle_error(1);
printf("After stopping the counters: %lld\n", values[0]); }

```

Possible output:

```

After reading the counters:
441027 After adding the counters:
891959 After stopping the
counters: 443994

```

The fully programmable low-level interface provides more sophisticated options for controlling the counters, such as setting thresholds for interrupt on overflow, as well as access to all native counting modes and events. Such an interface is intended for third-party tool writers or users with more sophisticated needs.

The PAPI specification also provides access to the most accurate timers available on the platform in use. These timers can be used to obtain both real and virtual time on each supported platform: the real time clock runs all the time (e.g., a wall clock), while the virtual time clock runs only when the processor is running in user mode.

In the following code example, `PAPI_get_real_cyc()` and `PAPI_get_real_usec()` are used to obtain the real time it takes to create an event set in clock cycles and in microseconds, respectively [3, 4]:

```

#include <papi.h>
int main(){
    long long start_cycles, end_cycles, start_usec, end_usec;
    int EventSet = PAPI_NULL;
    if (PAPI_library_init(PAPI_VER_CURRENT) != PAPI_VER_CURRENT)
        exit(1);

    /*Create an EventSet */
    if (PAPI_create_eventset(&EventSet) != PAPI_OK)
        exit(1);

    /* Gets the starting time in clock cycles */
    start_cycles = PAPI_get_real_cyc();

    /* Gets the starting time in microseconds */
    start_usec = PAPI_get_real_usec();

do_some_work();

    /* Gets the ending time in clock cycles */
    end_cycles = PAPI_get_real_cyc();

    /* Gets the ending time in microseconds */
    end_usec = PAPI_get_real_usec();
    printf("Wall clock cycles: %lld\n", end_cycles - start_cycles); printf("Wall
    clock time in microseconds: %lld\n", end_usec - start_usec);
}

```

Possible output:

```

Wall clock cycles: 100173

```

Wall clock time in microseconds: 136